

UNIX & LINUX FUNDAMENTALS

Technical Reference Document

Created on: 26 June 2026

UNIT 1: Introduction

1. Introduction to UNIX

UNIX is a multi-user and multi-tasking operating system that was developed in 1969 at Bell Laboratories (Bell Labs) by Ken Thompson, Dennis Ritchie, and their team. It was originally created to provide a simple, secure, and efficient environment for running programs and managing computer resources.

A multi-user operating system means that many users can use the same computer at the same time by logging into different accounts. A multi-tasking operating system means that multiple programs can run simultaneously without affecting each other. UNIX is widely used in universities, research organizations, banks, government offices, and large companies because of its stability and reliability.

UNIX acts as a bridge between the user and the computer hardware. Instead of communicating directly with the hardware, users interact with UNIX using commands or applications. UNIX manages memory, files, processors, and devices efficiently.

Features of UNIX

- Multi-user operating system
- Multi-tasking operating system
- Strong security using user accounts and permissions
- Portable (can run on different hardware platforms)
- Stable and reliable
- Hierarchical file system
- Powerful command-line interface called the Shell

Real-Life Example: Imagine a college computer laboratory with 40 students using one powerful server. Every student has a separate username and password. Each student can run programs, save files, and work independently. UNIX manages all users simultaneously without mixing their data.

Advantages

- High security
- Excellent performance
- Efficient resource management
- Stable for long-running systems
- Supports networking

Disadvantages

- Command-line interface may be difficult for beginners.
- Some UNIX versions are commercial and expensive.

Exam Definition: UNIX is a multi-user, multi-tasking operating system developed at Bell Labs in 1969. It provides a secure, stable, and efficient environment for managing hardware resources, files, users, and applications.

2. Discuss About POSIX

POSIX stands for Portable Operating System Interface. It is a set of international standards developed by the IEEE (Institute of Electrical and Electronics Engineers). The main purpose of POSIX is to make different UNIX operating systems behave in a similar way.

Before POSIX, different UNIX vendors used slightly different system calls, commands, and programming interfaces. Because of these differences, software written for one UNIX system often could not run on another UNIX system without modification.

POSIX solves this problem by defining standard rules for system calls, shell commands, utilities, file handling, process management, and programming interfaces. If an operating system follows POSIX standards, software developers can create one application that runs on many POSIX-compliant systems with little or no changes.

POSIX also defines standards for the C programming language interface used by UNIX applications.

Main Objectives of POSIX

- Improve software portability

- Standardize UNIX system calls
- Standardize shell commands
- Reduce software modification
- Improve compatibility between UNIX systems

Real-Life Example: Suppose a company develops payroll software on one UNIX system. Because both systems follow POSIX standards, the same software can easily run on another UNIX system without rewriting the entire program.

Advantages

- Software portability
- Standard programming interface
- Easy maintenance
- Better compatibility
- Reduces development cost

Exam Definition: POSIX (Portable Operating System Interface) is an IEEE standard that defines a common interface for UNIX operating systems so that programs can run on different UNIX systems with minimal changes.

3. Discuss About Linux (Syllabus Topic)

Linux is an open-source operating system created by Linus Torvalds in 1991. It was developed as a UNIX-like operating system and follows many UNIX principles. Although Linux behaves similarly to UNIX, it is not the original UNIX operating system.

Unlike traditional UNIX systems, Linux is freely available under the GNU General Public License (GPL). Anyone can view, modify, and distribute its source code. Because of this, Linux became popular among developers, educational institutions, and businesses.

Linux is widely used in servers, cloud computing, embedded systems, mobile devices, and supercomputers.

Features

- Open source

- Multi-user
- Multi-tasking
- Secure
- Portable
- Supports networking
- Stable

Applications

- Web servers
- Database servers
- Mobile devices
- Cloud computing
- Software development

Real-Life Example: Many websites, cloud services, and large internet companies use Linux-based servers because they are reliable and cost-effective.

Advantages

- Free to use
- Secure
- Stable
- Flexible
- Large community support

Disadvantages

- Different distributions may behave differently.
- Some commercial software is unavailable.

Exam Definition: Linux is an open-source UNIX-like operating system developed by Linus Torvalds in 1991. It provides a secure, multi-user, and multi-tasking environment and is widely used in servers and cloud computing.

4. Most Popular Distributions of Linux

A Linux distribution (Distribution or Distro) is a complete operating system built using the Linux kernel along with system utilities, software, package managers, and graphical interfaces.

Different distributions are designed for different users and purposes.

Popular Linux Distributions

1. Ubuntu

- Beginner-friendly
- Easy installation
- Popular for desktops and servers

2. Debian

- Very stable
- Highly secure
- Used in enterprise environments

3. Fedora

- Developed with modern technologies
- Frequently updated
- Used for software development

4. Red Hat Enterprise Linux (RHEL)

- Commercial distribution
- Enterprise-level support
- Used by large companies

5. CentOS (Traditional Enterprise Use)

- Community version based on Red Hat
- Stable for servers

6. SUSE Linux Enterprise

- Used in enterprise environments
- Strong security and administration tools

Real-Life Example: A software company may use Ubuntu for developers' computers, Red Hat Enterprise Linux for company servers, and Debian for database systems because each distribution is designed for different needs.

Advantages

- Different distributions for different purposes
- Regular updates
- Strong security
- Open-source ecosystem

Exam Definition: Linux distributions are different versions of Linux developed for specific purposes by combining the Linux kernel with software packages and utilities.

5. Compare Between UNIX and Linux

UNIX	Linux
Developed in 1969	Developed in 1991
Created at Bell Labs	Created by Linus Torvalds
Mostly commercial	Mostly open source
Source code usually closed	Source code freely available
Used in enterprise systems	Used in desktops, servers, cloud, mobile
Certified UNIX operating system	UNIX-like operating system
Vendor-specific versions	Many community distributions
Expensive licenses	Mostly free

Similarities

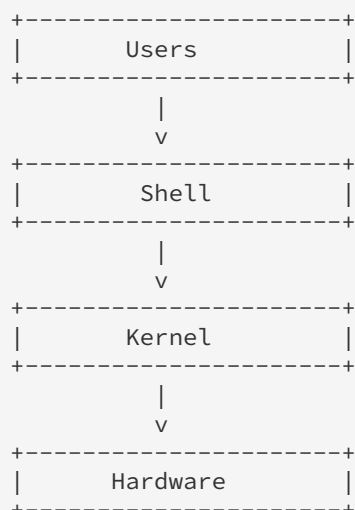
- Multi-user
- Multi-tasking

- Hierarchical file system
- Strong security
- Command-line interface
- Networking support

Real-Life Example: Large banks often use certified UNIX systems for mission-critical applications, while startups commonly use Linux because it is free and flexible.

6. UNIX System Architecture

UNIX architecture is divided into three main layers:



1. **User:** The user gives commands through the keyboard or applications.
2. **Shell:** The Shell acts as a command interpreter. It receives user commands and sends them to the kernel.
3. **Kernel:** The kernel is the heart of UNIX. It controls memory, CPU, files, devices, and processes.
4. **Hardware:** Includes CPU, RAM, disks, printers, keyboard, and other physical devices.

Real-Life Example: Think of a restaurant:

- User → Customer
- Shell → Waiter
- Kernel → Chef
- Hardware → Kitchen equipment

The customer tells the waiter what food is needed. The waiter tells the chef, who prepares the food using kitchen equipment and sends it back.

7. Discuss About the UNIX Kernel and System Call Interface

The Kernel is the core part of the UNIX operating system. It always remains in memory and manages all hardware resources.

The kernel performs many important functions such as:

- Process management
- Memory management
- File management
- Device management
- Security
- Scheduling

Users cannot directly access the kernel. Instead, programs communicate with it using System Calls.

A System Call Interface is a collection of functions that allows user programs to request services from the kernel.

Common System Call Categories

- Process Control
- File Management
- Device Management
- Memory Management

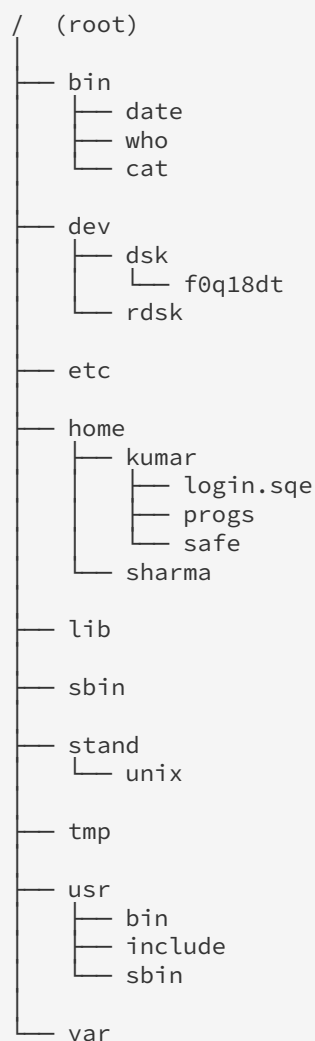
- Communication
- Information Management

Real-Life Example: Imagine a customer wants money from a bank locker. The customer cannot enter the locker room directly. The customer submits a request to the bank officer. The officer accesses the locker and returns the required item.

Here: Customer → User Program | Officer → System Call Interface | Locker → Kernel

8. UNIX Directory Structure

UNIX stores all files and directories in a hierarchical tree structure.



UNIT 2: UNIX COMMANDS

1. Create Directory

Command: `mkdir`

Work: Creates a new, empty directory.

```
mkdir projects
```

Creates a directory named `projects` in the current location.

```
mkdir -p projects/src/java
```

The `-p` flag creates parent directories as needed — if `projects` and `src` don't exist yet, it creates the whole path in one go.

2. Change Directory

Command: `cd`

Work: Changes your current working directory to another location.

```
cd /home/indra/projects
```

Moves you into the `projects` folder using an absolute path.

```
cd ..
```

Moves you up one level (to the parent directory).

```
cd
```

With no argument, `cd` alone takes you straight back to your home directory.

3. Move and Copy Directory

Move – Command: `mv`

Work: Moves a directory (or file) from one location to another; also used to rename.

```
mv projects /home/indra/work/
```

Moves the projects directory into /home/indra/work/.

Copy – Command: `cp`

Work: Copies a file or directory. For directories, you must use the `-r` (recursive) flag, since `cp` alone only copies files.

```
cp -r projects /home/indra/backup/
```

Recursively copies the entire projects directory (and everything inside it) into backup.

4. Create File

Command: `cat` (using redirection) or `touch`

```
cat > notes.txt
```

Opens a new file for input — type your content, then press Ctrl+D to save and exit.
Creates notes.txt with whatever you typed.

```
touch report.txt
```

Creates an empty file named report.txt (covered in more detail later in this unit).

5. Remove File

Command: `rm`

Work: Deletes (removes) a file permanently — Unix has no "Recycle Bin," so this is irreversible.

```
rm notes.txt
```

Deletes notes.txt immediately.

```
rm -i report.txt
```

The `-i` flag asks for confirmation before deleting — a safer habit.

6. Remove Directory

Command: `rmdir` (for empty directories) or `rm -r` (for non-empty)

```
rmdir oldfolder
```

Removes oldfolder only if it's empty. Fails with an error if it still contains files.

```
rm -r oldfolder
```

Recursively removes the directory and everything inside it, even if it's not empty. Use with caution — irreversible.

7. Listing Directory Information

Command: `ls`

Work: Lists the contents (files/subdirectories) of a directory.

```
ls
```

Lists files/folders in the current directory.

```
ls -l
```

Long listing format — shows permissions, owner, group, size, and modification date for each file.

```
ls -a
```

Shows all files, including hidden ones (those starting with .).

8. Types of Files in Unix

Unix recognizes several file types, identifiable via the first character in `ls -l` output:

Symbol	Type	Example
-	Ordinary/regular file	Text files, programs, C source code
d	Directory file	A folder
l	Symbolic link	A shortcut/pointer to another file
b	Block special file	Disk devices (/dev/dsk)
c	Character special file	Terminal, keyboard devices
p	Named pipe (FIFO)	Inter-process communication channel

Example: running `ls -l` and seeing:

```
-rwxr-xr-x  1 indra users  204 Jun 20 notes.txt
drwxr-xr-x  2 indra users  512 Jun 20 projects
```

The leading - tells you notes.txt is a regular file; the leading d tells you projects is a directory.

9. Change File Security (Permissions)

Command: `chmod`

Work: Changes the read, write, execute permissions of a file/directory for owner, group, and others.

```
chmod 755 script.sh
```

Sets permission so the owner can read/write/execute (7), while group and others can only read/execute (5).

```
chmod u+x script.sh
```

Symbolic mode – adds execute permission for the user/owner only.

Quick reference for permission numbers: 4 = read, 2 = write, 1 = execute (add them up: 7 = read+write+execute)

10. Creating Empty Files & Changing Timestamp

Command: `touch`

Work: Creates a new empty file if it doesn't exist; if it already exists, it just updates its timestamp (without changing content).

```
touch newfile.txt
```

Creates newfile.txt as an empty file (0 bytes) if it doesn't already exist.

```
touch existingfile.txt
```

If existingfile.txt already exists, this command leaves its content untouched but updates its last-modified timestamp to the current time.

11. Access Time, Modification Time, Change Time

Every Unix file has 3 separate timestamps, stored in its inode:

Timestamp	Meaning	Updated when...
Access time (atime)	Last time the file's content was read	You cat, open, or read the file
Modification time (mtime)	Last time the file's content was changed	You edit/write to the file
Change time (ctime)	Last time the file's metadata changed	Permissions, ownership, or filename changes

Example:

```
ls -lu file.txt      → shows access time
ls -l file.txt       → shows modification time (default)
ls -lc file.txt      → shows change time
```

This distinction is a classic exam question – remember: **mtime** = content changed, **ctime** = metadata changed, **atime** = just read/viewed.

12. Touch with -d and -t Options

-t option – set a specific timestamp directly:

```
touch -t 202506261230 file.txt
```

Sets file.txt's timestamp to 26 June 2026, 12:30 PM (format: YYYYMMDDhhmm).

-d option – set timestamp using a human-readable date string:

```
touch -d "2026-06-20 10:00:00" file.txt
```


Sets the timestamp to 20 June 2026, 10:00 AM, using a more flexible date format than -t.

(Note: exact -d/-t string parsing can vary slightly across Unix flavors — always check man touch on your specific system if it behaves unexpectedly.)

13. Finding Files and Directories

Command: `find`

Work: Searches the directory tree for files/directories matching given criteria.

```
find /home/indra -name "*.txt"
```

Searches under /home/indra for all files ending in .txt.

```
find . -type d -name "projects"
```

Searches the current directory (.) recursively for a directory (`-type d`) named projects.

```
find / -size +1000k
```

Finds files larger than 1000 KB anywhere on the system.

14. Soft Link and Hard Link

Command: `ln`

Hard Link:

```
ln original.txt hardlink.txt
```

Creates a hard link — hardlink.txt points to the same inode as original.txt. Both names are equally "real"; deleting one doesn't affect the other's data as long as one link remains.

Soft Link (Symbolic Link):

```
ln -s original.txt softlink.txt
```

Creates a symbolic link — softlink.txt is a separate file that just stores the path to original.txt. If you delete original.txt, the soft link becomes "broken" (dangling).

Key exam distinction:

Hard Link	Soft Link
Points to Same inode	Points to Path/filename of target
Survives original deletion? Yes (data still exists)	Survives original deletion? No (becomes broken)
Can link across filesystems? No	Can link across filesystems? Yes
Can link to a directory? No	Can link to a directory? Yes

15. About inode

An inode (index node) is a data structure that stores all metadata about a file — except its name. This includes:

- File type and permissions
- Owner and group ID
- File size
- Timestamps (atime, mtime, ctime)
- Pointers to the actual data blocks on disk

Important concept: The filename you see in a directory is just a label pointing to an inode number — it's not stored in the inode itself. That's exactly why mv (rename) is instant: it just changes which name points to which inode, without touching the actual data.

```
ls -li file.txt
```

Displays the inode number of file.txt.

16. Size of File System

Command: `df`

Work: Reports disk space usage of mounted filesystems.

```
df -k
```

Shows filesystem sizes in kilobytes — total space, used space, available space, and mount point.

Command: `du`

Work: Reports disk usage of files/directories specifically (how much space a folder is consuming).

```
du -s /home/indra
```

Shows the total size (summarized, `-s`) of everything inside `/home/indra`.

17. Use of > and >>

> — Output redirection (overwrite)

```
ls -l > filelist.txt
```

Sends the output of `ls -l` into `filelist.txt`, overwriting the file if it already exists.

>> — Output redirection (append)

```
echo "New entry" >> filelist.txt
```

Appends the text to the end of `filelist.txt`, without erasing existing content.

Key exam line: `>` destroys old content and replaces it; `>>` preserves old content and adds to it.

18. Use of Pipe and tee Command

Pipe – |

Work: Takes the output of one command and feeds it as the input to another command, allowing you to chain commands together.

```
ls -l | grep "txt"
```

Lists files, then pipes that output into grep, which filters and shows only lines containing "txt".

tee command

Work: Reads input and simultaneously writes it to a file and displays it on the screen – splits the output like a literal "T" junction in a pipe.

```
ls -l | tee filelist.txt
```

Shows the `ls -l` output on your screen and saves the exact same output into filelist.txt at the same time.

```
ls -l | tee -a filelist.txt
```

The `-a` flag makes tee append to the file instead of overwriting it.